Week 01: Introduction to Software Engineering

ICS 613

Software is everywhere

Society is dependent on software-intensive systems

Software plays a crucial role in our daily lives and is embedded in almost every aspect of modern society

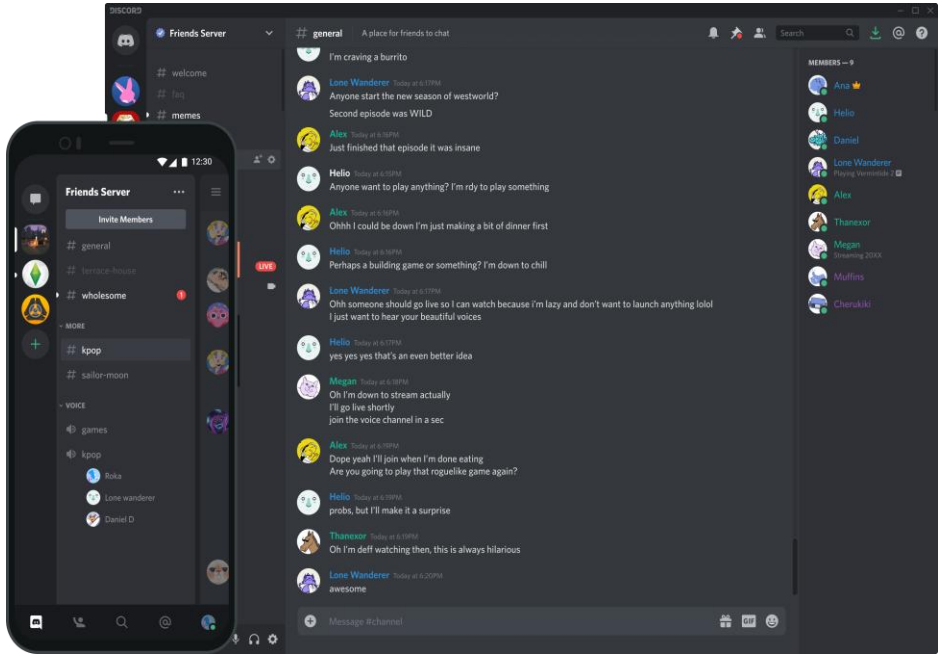
As humans advance, more advanced software is required to address new challenges (e.g., climate change)

Can you think of an industry that can function without the need of software systems?

What is software?

Software is the vehicle for delivering a product

- **Instructions** - When executed, provide desired features, function, and performance
- **Data Structures** - Enable programs to manipulate information adequately
- **Documentation** - Describes the operation and use of the programs



Discord: An example of how software serves as the vehicle for delivering seamless communication and community engagement.

Why is it “engineered”?

Software engineering is a systematic approach to software development within cost, time, and other constraints

Goes beyond coding: includes design, testing, maintenance

Balances creativity with discipline

Software engineering leads to a product that is reliable, efficient, and effective at what it does

IEEE Definition:
The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software

Why Software Engineering Matters

- Software is critical infrastructure in modern life
- Failures can have severe consequences (safety, finance, privacy)
- Demand for reliable, maintainable, scalable systems
- Teams, not individuals, build modern software

Common Problems:

- Projects over budget
- Late delivery
- Poor quality
- Difficult to maintain
- Security vulnerabilities
- User dissatisfaction

Software Engineer vs Computer Scientist vs Programmer vs Vibe Coder

Software Engineer

- Building systems
- Team collaboration
- Process and methodology
- Long-term maintenance
- Production systems
- Practical solutions

Computer Scientist

- Theory and research
- Mathematical foundations
- Algorithms and complexity
- Academic focus
- Innovation and discovery
- Computational thinking

Programmer

- Writing code
- Individual activity
- Short-term focus
- Quick solutions
- Personal projects

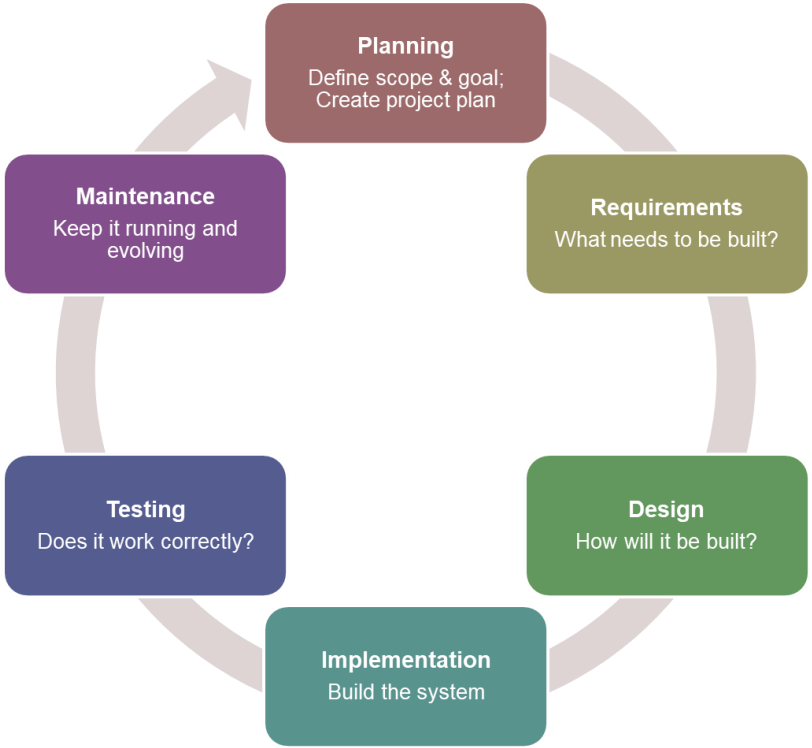
Vibe Coder

- No/limited background in software engineering
- AI-assisted development
- Prompt engineering
- Intuition-driven design
- Creative experimentation
- Iterative exploration

Software Development Life Cycle

SDLC

A framework that describes the different stages involved in the development of a software product, from the initial planning and requirements gathering phase, through the final testing and deployment phase.

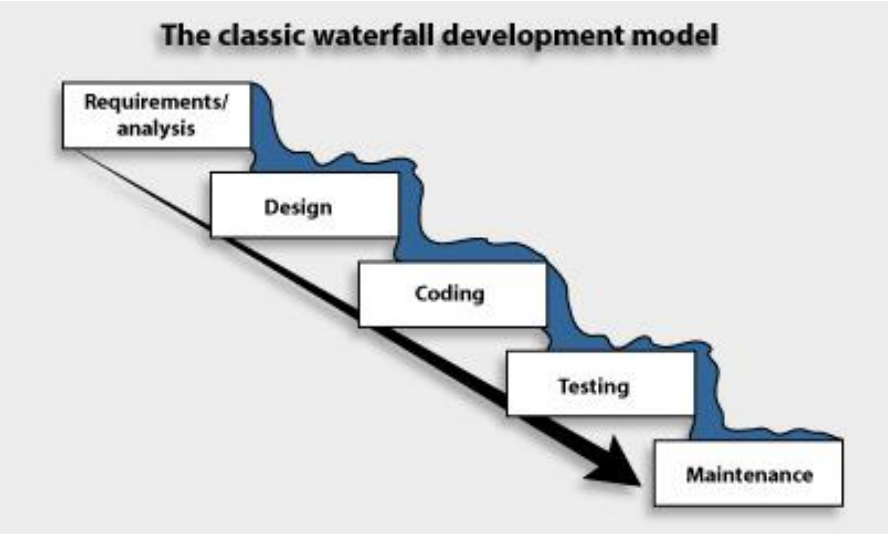


There are many different SDLC models

Some common SDLC models include:

- Waterfall: A linear, sequential approach to software development
- V-Model: A modified version of the Waterfall model, where each stage of the development process has a corresponding testing stage
- Agile: An iterative, incremental approach to software development
- Scrum: An Agile framework for managing and completing complex projects.
- Rapid Application Development (RAD): An Agile approach to software development that emphasizes rapid prototyping
- Rational Unified Process (RUP): An iterative and incremental process that is based on the Unified Modeling Language and the Unified Process framework
- OpenUP: An extension of RUP intended to be more lightweight and easy to use
- …

Waterfall



- **Linear, sequential flow:**

Each phase must complete before the next begins

- **Heavy upfront planning:**

All requirements are gathered and documented before any coding starts

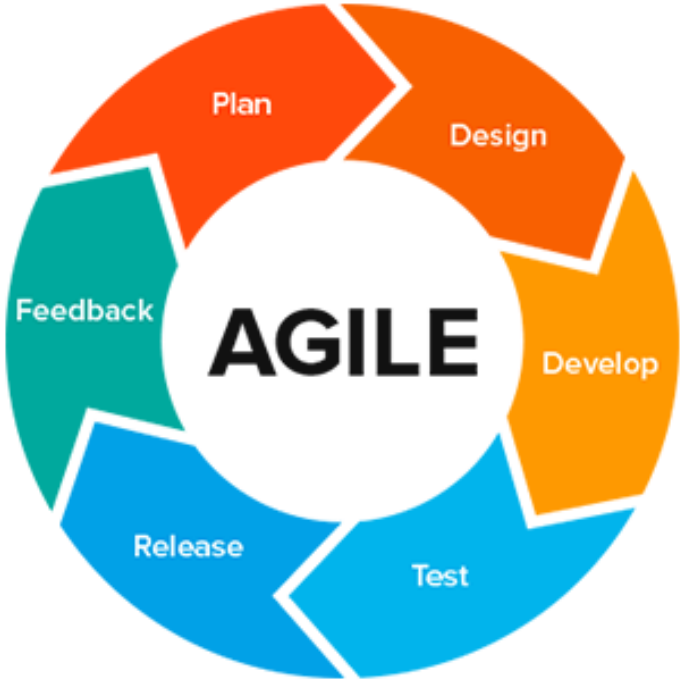
- **Best suited for**:

Well-defined, stable projects where requirements are unlikely to change

- **Key drawback:**

Late discovery of issues; working software isn't available until near the end

Agile



- **Iterative and incremental:**

software is built in small, usable chunks over short cycles (sprints)

- **Embraces changing requirements**:

adapts to feedback continuously rather than locking specs upfront

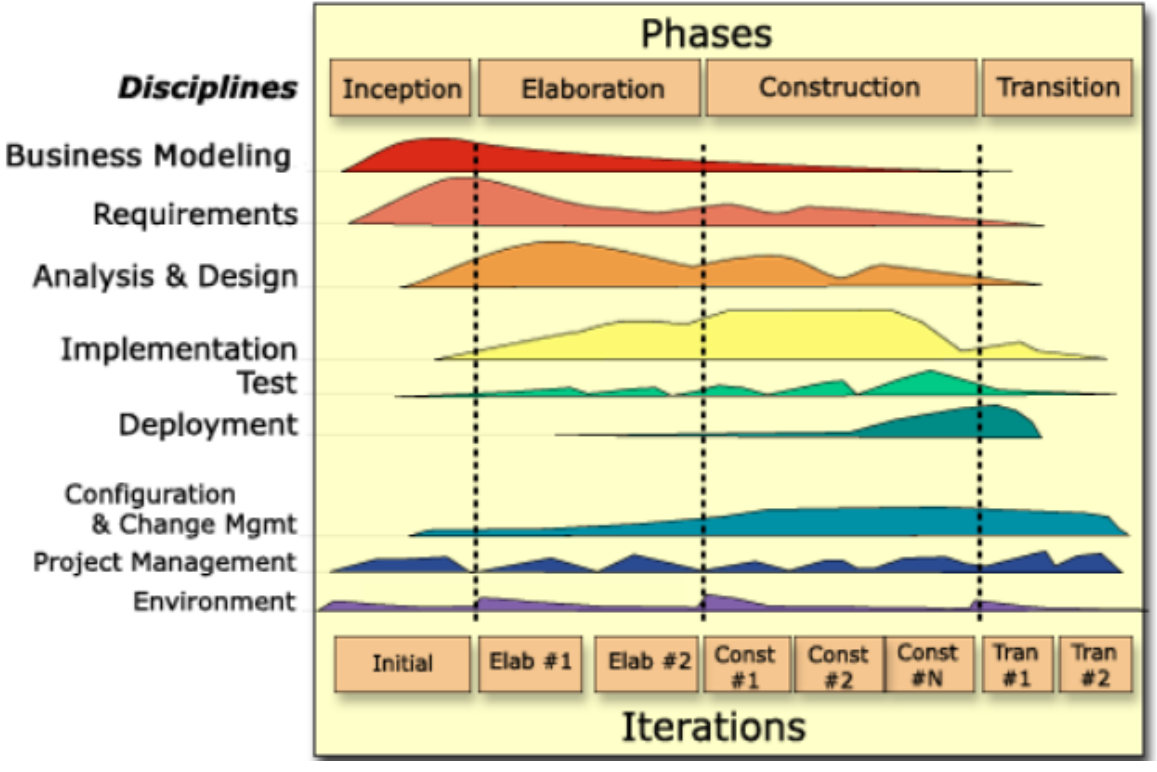
- **Customer collaboration**:

is prioritized over rigid contracts and documentation

- **Working software:**

is delivered frequently, often every 1–4 weeks

Rational Unified Process



- Process framework with defined phases
- Four lifecycle phases: Inception, Elaboration, Construction, Transition
- Architecture + risk stabilized early
- Artifacts, roles, disciplines, and milestones are explicit
- Can become documentation-heavy when applied rigidly

Is RUP the same as Agile?

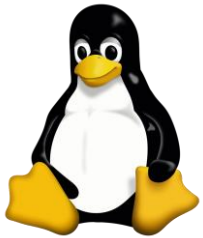
- RUP is a specific process framework; Agile is a philosophy and family of methods

  (Scrum, XP, Kanban, Lean, Crystal, and others are specific ways to apply Agile)
- RUP follows a defined lifecycle (Inception, Elaboration, Construction, Transition); Agile uses repeating sprints with less formal phases
- RUP emphasizes upfront planning, architecture, and risk analysis; Agile favors adaptive planning as requirements evolve
- RUP relies on heavier artifacts (use cases, architecture docs, models); Agile prefers lighter documentation and working software
- RUP is structured, disciplined, and sometimes heavyweight; Agile is lightweight, adaptive, and collaborative

Software maintenance

- Every software system undergoes maintenance
  - corrective, adaptive, preventive, perfective
- The costliest part of the software development lifecycle
  - consumes 60% - 80% of organization resources [1] [2]
- As an application evolves, it grows in size
  - more files/classes → more LOC

The Windows Calculator app has over **40,000 lines of code** [3]



Linux 4.7 is comprised of almost **22 million lines of code** [4]

[1] E. Burch and H. K. Hsiang-Jui Kungs, "Modeling software maintenance requests: a case study," 1997, ICSM
[2] L. Erlikh, "Leveraging legacy system dollars for e-business," in IT Professional, 2000.
[3] Count LOC online - https://api.codetabs.com/v1/loc?github=microsoft/calculator
[4] O. Schwahn, S. Winter, N. Coppik and N. Suri, "How to Fillet a Penguin: Runtime Data Driven Partitioning of Linux Code," in IEEE TDSE

Questions?